

Prueba de Caja Blanca

“Tilín Tilín Planeta Fiesta”

Integrantes:

Mateo Jaramillo

Valeria Sosa

Mery Velasquez

Miguel Verdezoto

Fecha: 2026/07/7

Requisito funcional N°9: Gestión de eventos

1. CÓDIGO FUENTE para gestion de eventos

```
private void crearServicio() {
    try {
        String tipoGrupo = this.vista.getComboBoxTipoGrupo().getSelectedItem().toString().trim();
        String nombreServicio = this.vista.getTxtNombreServicio().getText().trim();
        String precioStr = this.vista.getTxtPrecioServicio().getText().trim();
        String descripcion = this.vista.getTxtDescripcionServicio().getText().trim();

        // [Nodo 1] - Validación de Campos Vacíos
        if (tipoGrupo.isEmpty() || nombreServicio.isEmpty() || precioStr.isEmpty() ||
            descripcion.isEmpty()) {
            vista.mostrarMensajeError("Todos los campos son obligatorios.", "Campos incompletos");
            return; // [Nodo 2] (Termina por campos vacíos)
        }

        float precio;
        try {
            // [Nodo 3] - Intento de parsear el precio a float
            precio = Float.parseFloat(precioStr);
        } catch (NumberFormatException ex) {
            // [Nodo 4] - Excepción si el formato de precio es inválido
            vista.mostrarMensajeError("El precio debe ser un número válido.", "Formato incorrecto");
            return; // [Nodo 5] (Termina por precio no numérico)
        }

        // [Nodo 6] - Validación de Precio Positivo
        if (precio <= 0) {
            vista.mostrarMensajeError("El precio debe ser mayor a cero.", "Precio inválido");
            return; // [Nodo 7] (Termina por precio menor o igual a cero)
        }

        // [Nodo 8] - Verificación de Duplicado en MongoDB
        Document servicioDuplicado = coleccionEventos.find(eq("nombreEvento",
            nombreServicio)).first();
        if (servicioDuplicado != null) {
            vista.mostrarMensajeError("Ya existe un servicio registrado con este nombre.", "Servicio
            duplicado");
            return; // [Nodo 9] (Termina por servicio duplicado)
        }

        // [Nodo 10] - Flujo de Inserción Exitosa en MongoDB
        int nuevoId = generarSiguieteId();
        Evento nuevoEvento = new Evento(nuevoId, tipoGrupo, nombreServicio, precio, descripcion);

        Document nuevoDoc = new Document("idEvento", nuevoEvento.getIdEvento())
            .append("tipoEvento", nuevoEvento.getTipoEvento())
            .append("nombreEvento", nuevoEvento.getNombreEvento())
            .append("precioEvento", (double) nuevoEvento.getPrecioEvento())
            .append("descripcionEvento", nuevoEvento.getDescripcionEvento());

        coleccionEventos.insertOne(nuevoDoc);
    }
}
```

```

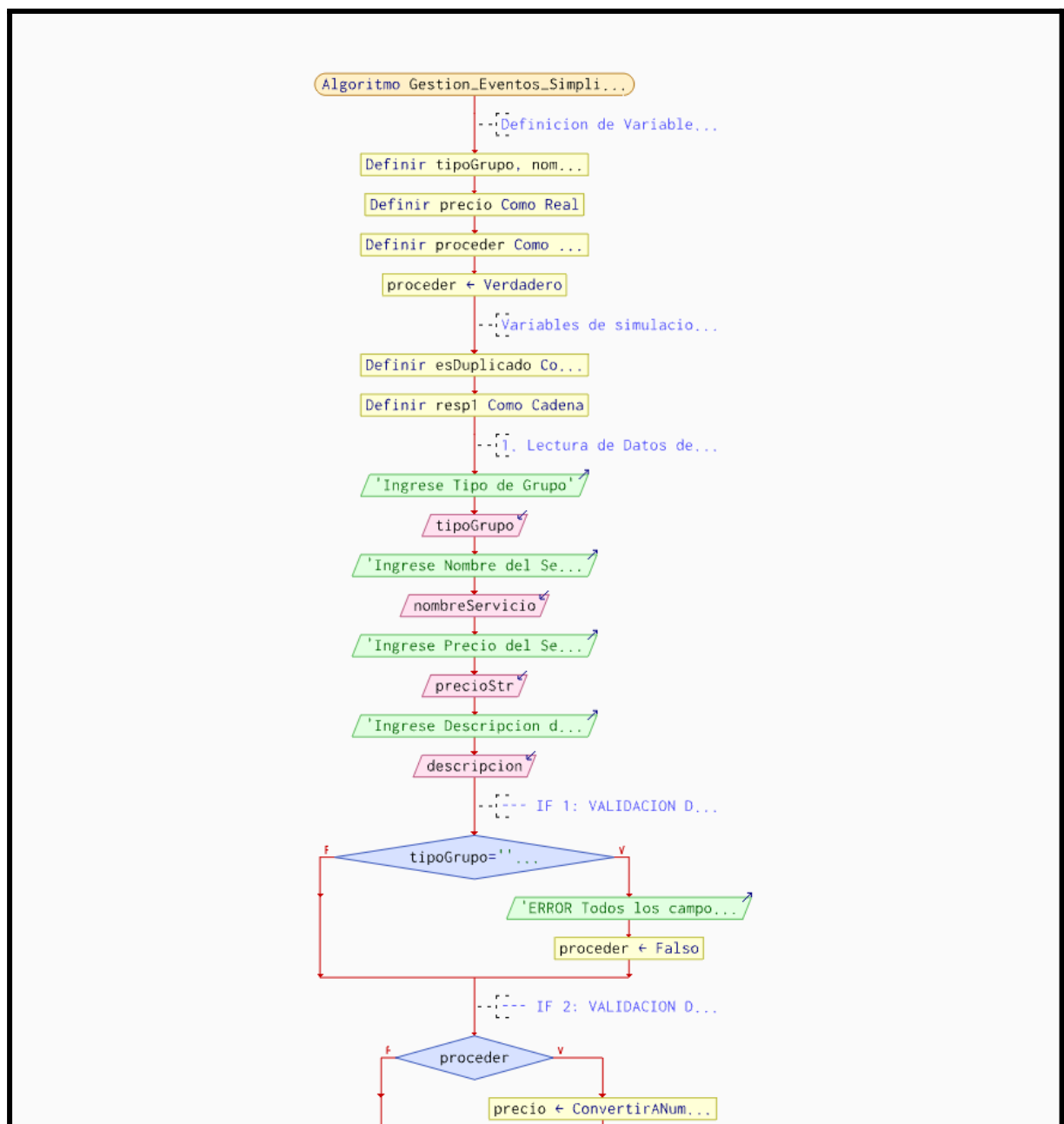
    vista.mostrarMensajeInformativo("Servicio del grupo " + tipoGrupo + " creado con éxito.",
    "Éxito");

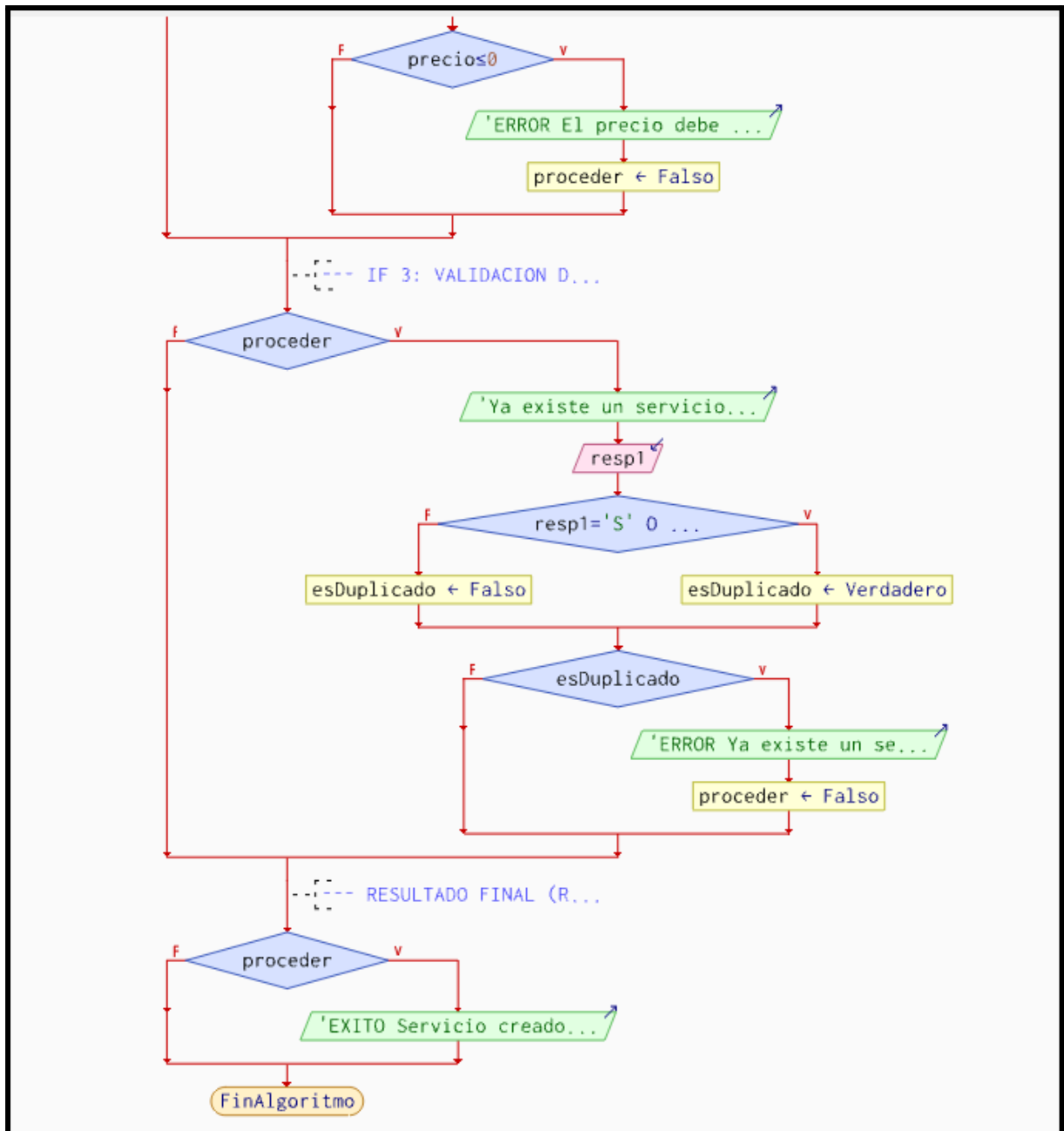
    actualizarTabla();
    // [Nodo 11] (Termina con éxito)

} catch (Exception ex) {
    // Captura de errores generales del sistema
    vista.mostrarMensajeError("Ocurrió un error inesperado al procesar los datos.", "Error");
}
// [Nodo 12] - Fin general del método (Punto común de salida)
}

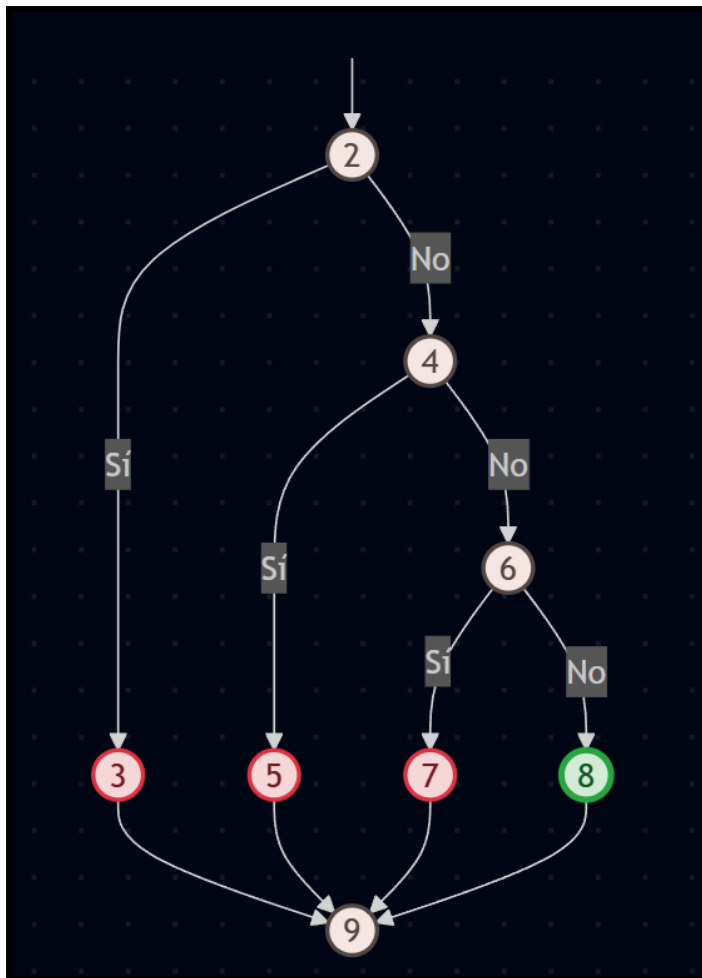
```

2. DIAGRAMA DE FLUJO (DF)





3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

- R1: 1 → 2 → 3 → 9 (Al menos un campo de texto está vacío)
- R2: 1 → 2 → 4 → 5 → 9 (El precio del servicio es menor o igual a cero)
- R3: 1 → 2 → 4 → 6 → 7 → 9 (El nombre del servicio ya está registrado en MongoDB)
- R5: 1 → 2 → 4 → 6 → 8 → 9 (Servicio del grupo creado y registrado con éxito)

5. COMPLEJIDAD CICLOMÁTICA I.

- Número de Nodos (N): 9
- Número de Aristas (A): 11
- Nodos Predicado (P): 3 (Nodos correspondientes a las decisiones de los condicionales **if**: Nodos 2, 4 y 6).

Método 1: Por nodos predicado

$$V(G) = P + 1$$

$$V(G) = 3 + 1$$

$$V(G) = 4$$

Método 2: Por aristas y nodos

$$V(G) = A - N + 2$$

$$V(G) = 11 - 9 + 2$$

$$V(G) = 4$$